hexens x sentio

Security Review Report for Sentio

March 2026

Table of Contents

1. About Hexens
2. Executive summary
3. Security Review Details
    - Security Review Lead
    - Scope
    - Changelog
4. Severity Structure
    - Severity characteristics
    - Issue symbolic codes
5. Findings Summary
6. Weaknesses
    - Incoming bridge mints are not blocked when contract is paused
    - Whitelisting check when minting can be optimized

1. About Hexens

Hexens is a pioneering cybersecurity firm dedicated to establishing robust security standards for Web3 infrastructure, driving secure mass adoption through innovative protection technology and frameworks. As an industry elite experts in blockchain security, we deliver comprehensive audit solutions across specialized domains, including infrastructure security, Zero Knowledge Proof, novel cryptography, DeFi protocols, and NFTs.

Our methodology combines industry-standard security practices combined with unique methodology of two teams per audit, continuously advancing the field of Web3 security. This innovative approach has earned us recognition from industry leaders.

Since our founding in 2021, we have built an exceptional portfolio of enterprise clients, including major blockchain ecosystems and Web3 platforms.

2. Executive Summary

This report covers the security review for Sentio. This audit covered the Solidity smart contracts of the Sentio Token. It consisted of bridgeable tokens that are compatible with LayerZero and Optimism L2s.

Our security assessment was a full review of the code, spanning a total of 1 day.

During our review, we did not identify any major or minor security vulnerabilities.

We did identify several code optimizations.

All reported issues were fixed or acknowledged by the development team and subsequently verified by us.

We can confidently say that the overall security and code quality have increased after completion of our audit.

3. Security Review Details

Review Led by

Kasper Zwijsen, Head of Audits

Scope

The analyzed resources are located on:

🔗 https://github.com/sentioxyz/sentio-token-audit/tree/9c7658a1975a35e26c500523e6b890bb30526a37/contracts

The issues described in this report were fixed in the following commit:

🔗 https://github.com/sentioxyz/sentio-token-audit/tree/30b0f4edbfa255f3adb6f2c44fdd622626ce527c

Changelog

10 March 2026	Audit start
11 March 2026	Initial report
13 March 2026	Revision received
16 March 2026	Final report

4. Severity Structure

The vulnerability severity is calculated based on two components:

1. Impact of the vulnerability
2. Probability of the vulnerability

Impact	Probability			
	Rare	Unlikely	Likely	Very likely
Low	Low	Low	Medium	Medium
Medium	Low	Medium	Medium	High
High	Medium	Medium	High	Critical
Critical	Medium	High	Critical	Critical

- **Severity Characteristics**

Smart contract vulnerabilities can range in severity and impact, and it's important to understand their level of severity in order to prioritize their resolution. Here are the different types of severity levels of smart contract vulnerabilities:

Critical

Vulnerabilities that are highly likely to be exploited and can lead to catastrophic outcomes, such as total loss of protocol funds, unauthorized governance control, or permanent disruption of contract functionality.

High

Vulnerabilities that are likely to be exploited and can cause significant financial losses or severe operational disruptions, such as partial fund theft or temporary asset freezing.

Medium	Vulnerabilities that may be exploited under specific conditions and result in moderate harm, such as operational disruptions or limited financial impact without direct profit to the attacker.
Low	Vulnerabilities with low exploitation likelihood or minimal impact, affecting usability or efficiency but posing no significant security risk.
Informational	Issues that do not pose an immediate security risk but are relevant to best practices, code quality, or potential optimizations.

- **Issue Symbolic Codes**

Each identified and validated issue is assigned a unique symbolic code during the security research stage.

Due to the structure of the vulnerability reporting flow, some rejected issues may be missing.

5. Findings Summary

Severity	Number of findings
Critical	0
High	0
Medium	0
Low	0
Informational	2
Total:	**2**

Informational

Fixed

Acknowledged

6. Weaknesses

This section contains the list of discovered weaknesses.

SENT1-1 | Incoming bridge mints are not blocked when contract is paused

Acknowledged

Severity:	Informational	Probability:	Very likely	Impact:	Informational

Path:

SentioTokenBSC.sol:_update#L85-L93

Description:

The **_update** function of the SentioTokenBSC contract has been overridden to block any transfers to and from non-whitelisted addresses when the contract has been paused.

It does allow for minting to be continued (where **from == address(0)**), which allows the continued usage of the privileged **mint** function.

However, since the contract inherits from OFT, this also has the consequence that bridge transfers can continue to work to non-whitelisted accounts when the contract is paused. This is because the OFT **_credit** function will call the internal **_mint** function to transfer the tokens to the recipient of the LZ message.

Outgoing bridge transfers would be blocked, as it would check the whitelist for both the **from** address and the **to** address (**address(0)**).

```
function _update(address from, address to, uint256 amount) internal override {
    if (paused()) {
        bool minting = from == address(0);
        bool whitelistTransfer = _isWhitelisted[from] && _isWhitelisted[to];
        require(minting || whitelistTransfer, "paused and not whitelisted");
    }


    super._update(from, to, amount);
}
```

Remediation:

If this is unintended, then these 2 types of mints should be handled separately such that bridge mints are blocked when the contract is paused.

Commentary from the client:

"If the pause will also stop OFT _creadit to call _mint on bsc, it will only stop the minting on BSC, but not the burning on the other chain, so this seems a very strange state. So we think it's better to let people see token transfered to BSC but not just be able to immedidately use when pausing.
Also pausing is only supposed to be used in the initial IDO phase, so in practice we will not pause the second time."

SENT1-2 | Whitelisting check when minting can be optimized

Fixed

Severity:	Informational	Probability:	Very likely	Impact:	Informational

Path:

SentioTokenBSC.sol:_update#L85-L93

Description:

The SentioTokenBSC contract overrides the **_update** function to implement whitelisted transfers when the contract is paused. It does allow for minting and transfers from/to whitelisted addresses.

It checks for minting by comparing **from** with **address(0)** and then also reads from the **_isWhitelisted** storage mapping for both **from** and **to** to check for whitelisted status. Because it first stores both results in a boolean and only then for a **require** check, the code will performs the 1 or 2 SLOADs for the whitelist check no matter what.

If these were to be combined into 1 comparison statement, then the code execution could return early from the boolean expression and return **true** or **false** depending on the values.

```
function _update(address from, address to, uint256 amount) internal override {
    if (paused()) {
        bool minting = from == address(0);
        bool whitelistTransfer = _isWhitelisted[from] && _isWhitelisted[to];
        require(minting || whitelistTransfer, "paused and not whitelisted");
    }



    super._update(from, to, amount);
}
```

Remediation:

For example:

```
if (paused()) {
    require(from == address(0) || (_isWhitelisted[from] && _isWhitelisted[to]), "paused and not whitelisted");
}
```

hexens x sentio